МОНГОЛ УЛСЫН ИХ СУРГУУЛЬ
МЭДЭЭЛЛИЙН ТЕХНОЛОГИ, ЭЛЕКТРОНИКИЙН СУРГУУЛЬ
МЭДЭЭЛЭЛ, КОМПЬЮТЕРИЙН УХААНЫ ТЭНХИМ

Тооцоолол ба өгөгдлийн ухаан

#6 - Python программчлал

(object oriented programming)

Объект хандлагат программчлал

Объект

- Python олон төрлийн өгөгдлийн төрөлтэй:

```
256   3.14159   "Hello"   [1, 5, 7, 11, 13]
{"EN": "English", "MN": "Mongolian"}
```

- Эдгээр нь бүгд **объект**, объект бүр:
  - Дотоод **өгөгдлийн дүрслэл**тэй (анхдагч, нийлмэл)
  - Объект бүр **харилцаж** болох аргуудтай (procedure, method) байдаг
- Объект бол ямар нэг **төрлийн** (**type**) тухайн нэг **тохиолдол** (**instance**)
  - `256` нь `int` төрлийн instance
  - `"hello"` нь `str` төрийн instance

Объект хандлагат программчлал (OOP)

- **Python-ы бүх зүйл нь объект** (бас төрөлтэй)
- Объектыг ямар нэг төрлөөс **үүсгэж болно**
- Объектыг **өөрчилж болно**
- Объектыг **устгаж болно**

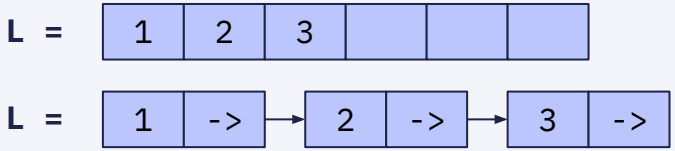
Объект гэж юу вэ?

Объект бол **өгөгдлийн хийсвэрлэл** дотроо...

- **Дотоод дүрслэлтэй** (**internal representation**)
  - Өгөгдлийн атрибут (attributes)-уудаар
- Объекттой харьцах **интерфейс**тэй (**interface**)
  - Method-р дамжуулж (ө.х. функц)
  - Хэрэгжүүлэлтийг нууж, зан чанарыг тодорхойлно

Жишээ: list төрлийн [1, 2, 3, 4] объект

- (1) Жагсаалт **дотроо яаж дүрслэгддэг** вэ?
  Хэрэглэгч бид нарт энэ тийм ч чухал биш



- (2) Жагсаалттай хэрхэн **харьцаж, өөрчлөх** вэ?
  - `L[i], L[i:j], +`
  - `len(), min(), max(), del(L[i])`
  - `L.append(), L.extend(), L.count(), L.index(), L.insert(), L.pop(), L.remove(), L.reverse(), L.sort()`

- Дотоод дүрслэл нь далд байх
- Дотоод дүрслэлээр нь оролдвол зан авир нь өөрчлөгдөх эрсдэлтэй

Бодит амьдрал дээрх жишээ

- **Цахилгаан шат**: давхар хооронд явдаг хайрцаг
  - Урт, өргөн, өндөр, даац, багтаамж, байрлаж буй давхар
  - Давхар хооронд явна, хүн нэмнэ, хасна
- **Ажилтан**: компанид ажилладаг хүн
  - Нэр, төрсөн өдөр, цалингаар тодорхойлогдоно
  - Нэр, цалин өөрчилж болно
- **Дэлгүүрийн оочир**: хамгийн эхэнд зогсож буйд эхлээд үйлчилнэ
  - Үйлчлүүлэгчдийг нэрийн жагсаалтаар дүрсэлнэ
  - Жагсаалтын төгсгөлд шинэ үйлчлүүлэгч нэмж, эхнээс нь хасна
- **Овоолсон бин/гамбир**: Хамгийн эхэнд хийснийг хамгийн сүүлд иднэ
  - Овоолгыг жагсаалтаар илэрхийлж болно
  - Шинэ бин жагсаалтын төгсгөлд нэмэгдэнэ, авахдаа төгсгөлөөс авна

OOP давуу тал

- Тодорхой аргаар **багц өгөгдөл** дээр ажиллах боломжтой
- Хөгжүүлэлтийн **хуваан-нэгтгэх** (**divide-and-conquer**) арга
- **Класс** нь кодыг **дахин ашиглах**ад хялбар болгоно
  - Python модулиуд шинэ класс тодорхойлдог
  - Класс бүр бие даасан орчин (функц, хувьсагчийн нэр давхцахгүй)
  - Удамшлын (inheritance) механизмын тусламжтай классыг цааш өргөтгөх, дахин тодорхойлох, дэд класс үүсгэж болно

Класс тодорхойлох нь загварчлалын шийдвэр гаргаж байгаа хэлбэр

Класс ямар өгөгдөл агуулахыг чи шийднэ.
Хэрэглэгч классаар юу хийж болохыг чи шийднэ.

Классаар шинэ өгөгдлийн төрөл үүсгэх

- **Класс үүсгэх** болон классын **instance хэрэглэх**ийн ялгааг ойлгох нь чухал
- Класс **үүсгэх** нь:
  - Классын нэрийг тодорхойлох
  - Классын атрибутыг тодорхойлох
- Класс **хэрэглэх** нь:
  - Классын **instance** үүсгэх
  - Instance дээр үйлдэл гүйцэтгэх

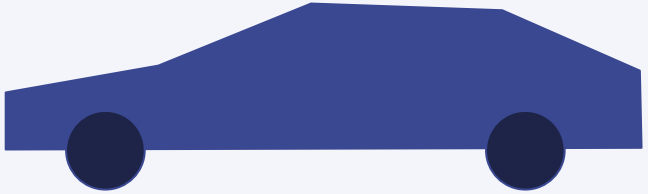
Жишээ: Машин

Класс

- **Шинж чанар:**
  - Өнгөтэй
  - Үнэтэй
  - Загвартай
  - Хөдөлгүүртэй
- **Үйлдэл:**
  - start()
  - forward()
  - backward()
  - stop()

Instance үүсгэх

Объект (instance)

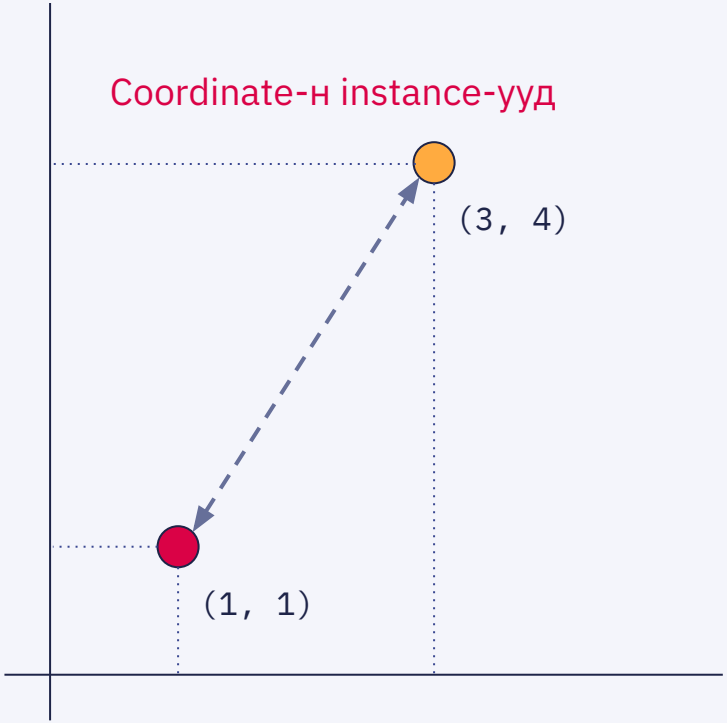


- **Шинж чанарыг илэрхийлсэн утга:**
  - Өнгө: Хөх
  - Үнэ: $30,000
  - Загвар: Audi
  - Хөдөлгүүр: 2,000
- **Үйлдэл:**
  - start()
  - forward()
  - backward()
  - stop()

Функцтэй адилтгах нь

- **Класс тодорхойлох** нь функц тодорхойлохтой адилтгаж болно
  - Функцээр бид Python-д ийм процедур байгаа гэдгийг хэлж өгнө
  - Классаар бид Python-д ийм **өгөгдлийн төрлийн загвар** байгаа гэдгийг хэлж өгнө
    - Өгөгдлийн атрибут
    - Арга
- **Классын instance үүсгэх** нь функц дуудахтай адилтгаж болно
  - Функцийг өөр өөр параметрээр дуудаж болно
  - Классаас шинэ **объект санах ойд үүснэ**

Координат төрөл



- Ямар **өгөгдөл** объектыг дүрслэх эсэхийг шийдэх
  - 2 хэмжээст хавтгайд координатыг **x болно y утга**ар тодорхойлно
- Тухайн өгөгдөл дээр ямар **үйлдэл** хийж болохыг шийдэх
  - Координат нь x, y тэнхлэгээс хэр хол байгааг хэлэх
  - Хоёр цэгийн хоорондох **зай**г тооцоолох (Пифагор)

Шинэ төрөл тодорхойлох

- **`class`** түлхүүр үгийн тусламжтай шинэ төрөл тодорхойлно

class definition

нэр/төрөл

классын эцэг

```
class Coordinate(object):
    # классын атрибутууд
```

- Классын тодорхойлолт дотор юу байгааг хоосон зайгаар баруун тийш түлхэн тэмдэглэнэ, функцийн их бие бичдэгтэй адил
- **`Coordinate`** нь Python объект бөгөөд **`object`**-н бүх атрибутыг удамшиж авна (удамшлыг дараа нь дэлгэрүүлж үзнэ)

Атрибут (attribute) гэж юу вэ?

- **Класст харьяалагдах** өгөгдөл болон үйлдлүүд
  - **Data attributes**
    - Классыг бүрдүүлж буй бусад объект/хувьсагчаар илэрхийлэгдсэн өгөгдөл
    - *Жишээ нь координат нь хоёр тооноос бүрдэнэ*
  - **Methods** (procedural attributes)
    - Зөвхөн тухайн классын хувьд хэрэглэхэд зориулсан функц
    - Классын объекттой харилцах аргууд
    - *Жишээ нь хоёр координатын хоорондох зайг олж болох боловч хоёр жагсаалтын хоорондох зайг олох нь утгагүй юм*

Классын instance хэрхэн үүсгэхийг тодорхойлох

- Классын instance үүсгэхийн тулд **хэрхэн instance үүсгэх вэ** гэдгийг тодорхойлох шаардлагатай
- **`__init__` хэмээх тусгай method**-оор классын instance үүсэх үед шаардлагатай өгөгдөл авах, гүйцэтгэх шаардлагатай үйлдлүүдийг ажиллуулна

```
class Coordinate(object):
    def __init__(self, xval, yval):
        self.x = xval
        self.y = yval
```

Coordinate объект үүсгэхэд шаардлагатай өгөгдөл

Instance үүсгэх тусгай method, хоёр доогуур зураастай

Классын instance үүсгэлгүйгээр instance-г заах параметр

Бидний өгөгдлийн төрлийг бүрдүүлэх хоёр data attribute

- **`self`** нь **объектод харьяалагдах хувьсагч** үүсгэнэ
- **`self`** хэрэглээгүй бол зүгээр энгийн хувьсагч үүсгэнэ

Классыг тодорхойлж байх үед бидэнд ямар нэгэн бодит объект байхгүй

Зөвхөн тодорхойлолт.

```
class Coordinate(object):
    def __init__(self, xval, yval):
        self.x = xval
        self.y = yval
```

Классын instance үүсгэх нь

- self аргументыг Python автоматаар дамжуулна, бичих шаардлагагүй

```
c = Coordinate(3, 4)
origin = Coordinate(0, 0)
```

Coordinate төрлийн шинэ объект үүсгэж __init__ рүү 3, 4 утга дамжуулна

- Instance-н data attribute-г **instance variables** гэнэ
  - Data attribute нь self.XXX гэж тодорхойлогдох ба түүн рүү цэг тэмдэглэгээгээр хандах боломжтой
  - Бүх instance нь data attribute-тай, гэхдээ утга нь өөр!

```
print(c.x)
print(origin.x)
```

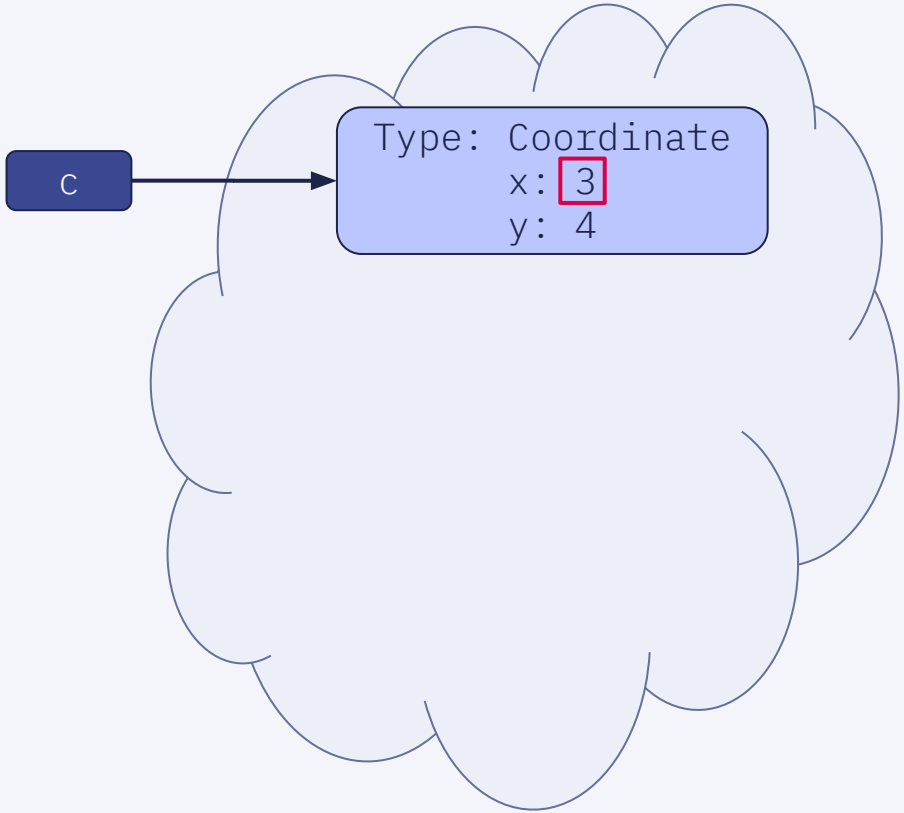
цэг тэмдэглэгээгээр c instance-н атрибут руу хандаж байна

Instance-г дүрсэлж үзье

- Координат төрлийн хувьсагч үүсгэсэн гэж үзье

```
c = Coordinate(3, 4)
```

- Санах ойд үүссэн нэг бүтэцлэгдсэн өгөгдөл гэж бодож болно
- **c.x** хэмээн дуудахад **c**-н зааж буй бүтцийг хайж олоод, дотроос нь **x**-г олно

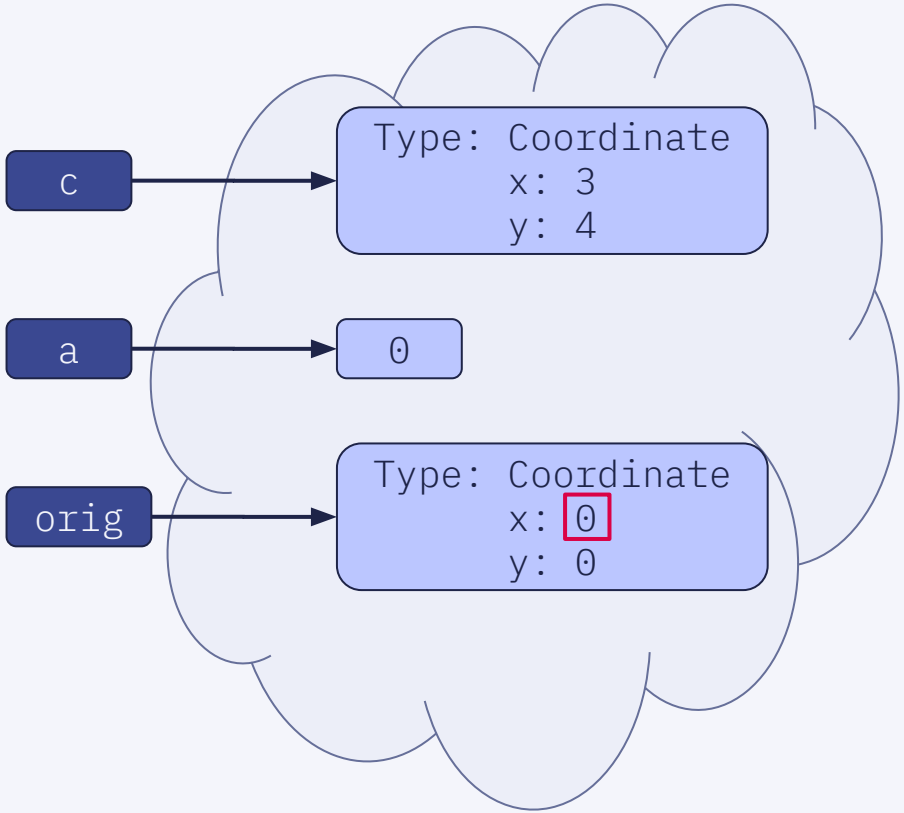


Instance-г дүрсэлж үзье

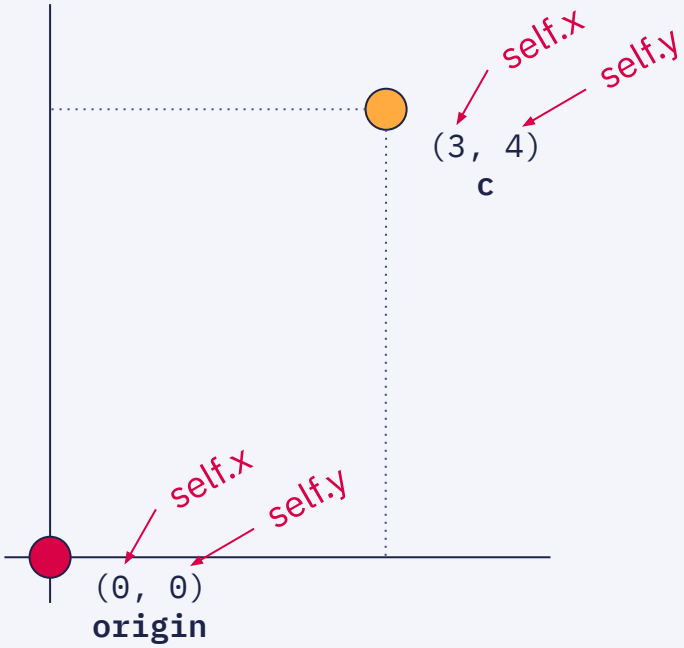
- Хувьсагчийн утга ашиглан өөр нэг instance үүсгэж үзье

```
a = 0
c = Coordinate(a, a)
orig.x
```

- Instance нь санах ойд байрлах объект
- Цэг нь тухайн санах ойд байгаа объектын атрибут руу л хандаж байгаа хэрэг



Instance-г дүрсэлж үзье



Coordinate төрлийн заргвар/тодорхойлолт (template)

```
class Coordinate(object):
    def __init__(self, xval, yval):
        self.x = xval
        self.y = yval
```

```
c = Coordinate(3, 4)
origin = Coordinate(0, 0)
print(c.x)
print(origin.x)
```

Coordinate төрлийн бодит объект (instance) үүсгэх

Method гэж юу вэ?

- Procedural attribute
  - **Зөвхөн тухайн класстай ажилладаг функц** гэж бодоход болно
- Python үргэлж method дуудсан объектыг функцийн эхний аргументаар дамжуулж өгнө
  - Бүх method-н эхний параметрийн нэрийг **`self`** хэмээн нэрлэхээр тогтсон

Coordinate класст method тодорхойлох

```
class Coordinate(object):
    def __init__(self, xval, yval):
        self.x = xval
        self.y = yval

    def distance(self, other):
        x_diff_sq = (self.x - other.x) ** 2
        y_diff_sq = (self.y - other.y) ** 2
        return (x_diff_sq + y_diff_sq) ** 0.5
```

энэ method-г дуудсан объектыг заахад хэрэглэнэ

method-н нөгөө параметр

цэгээр self-н x-д хандах

цэгээр other-н x-д хандах

- self болон цэгэн тэмдэглэгээнээс бусдаар method нь яг л функц шиг ажиллана (парам. авах, үйлдэл гүйцэтгэх, утга буцаах)

Method-г хэрхэн дуудах вэ?

- Ямар ч атрибут руу хандахдаа “**.**” **оператор**ыг хэрэглэнэ
  - Объектын өгөгдөл (өмнөх жишээний с.х)
  - Объектын гишүүн функц (method)
- Dot notation

```
<object_variable>.<method>(<parameters>)
```

method буюу гишүүн функцийг дуудах объект, классын тодорхойлолтод self нь болно

функцийн нэр

self-г оруулахгүй, self нь цэгийн урд байх объект

- Танил байна уу?
  - `my_list.append(4)`
  - `my_list.sort()`

Method хэрэглэх нь

- Хэрэглэхдээ:

```
c = Coordinate(3, 4)
orig = Coordinate(0, 0)
print(c.distance(orig))
```

method дуудаж буй объект

method-н нэр

self-с бусад параметр

Бидний классдаа тодорхойлсон distance method

```
def distance(self, other):
    x_diff_sq = (self.x - other.x) ** 2
    y_diff_sq = (self.y - other.y) ** 2
    return (x_diff_sq + y_diff_sq) ** 0.5
```

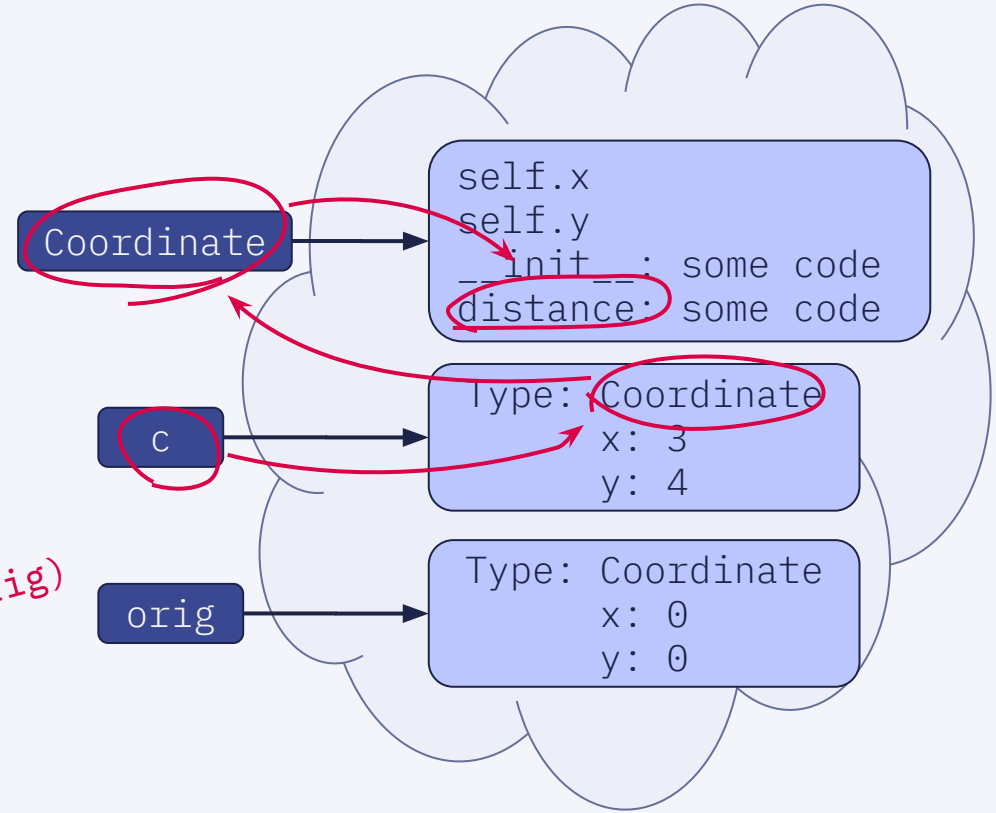
- `self` нь method-г дуудсан объект болж байгааг анзаараарай (цэгийн урд бичсэн хувьсагч)

Функц дуудалтыг дүрсэлж үзье

- Доорх кодыг ажиллуулсан гэж үзье

```
c.distance(orig)
```

1. Цэгийн өмнөх нь объект
2. **c**-н төрлийг хайна
3. Цэгийн дараах нь method
4. Объектын класст **distance**-г хайна
5. Олсон method-г дуудна
   c-г **self** параметрт
   orig-г **other** параметрт



Coordinate.distance(c, orig)

Method хэрэглэх нь

- **Хэвшсэн бичлэг:**

```
c = Coordinate(3, 4)
origin = Coordinate(0, 0)
c.distance(origin)
```

method дуудаж буй объект, distance-н self-р очино

method-н нэр

self-с бусад параметр (self нь с болно)

- **Байж болох:**

```
c = Coordinate(3, 4)
origin = Coordinate(0, 0)
Coordinate.distance(c, origin)
```

классын нэр (Coordinate төрлийн ОБЪЕКТ БИШ)

method-н нэр

функцийн параметрүүд, функцийг дуудах байсан объектыг мөн self рүү дамжуулж өгнө

`.` оператор өгөгдөл болон функц рүү хандана.

Data attribute-г `self.` Гэж тодорхойлно
Method нь класс дотор эхний параметрээрээ `self` авдаг функц

Класс хэрэгжүүлэх vs Класс хэрэглэх

Объектын шинэ төрлийг классаар **хэрэгжүүлэхдээ**:

- Класс **тодорхойлох**
- **Data attribute**-уудыг тодорхойлох (Объект нь ЮУ вэ?)
- **Method**-уудыг тодорхойлох (ЯАЖ объектыг хэрэглэх вэ?)

Классаар **нийтлэг** шинж, үйлдлийг хийсвэрээр дүрсэлнэ

Объектын шинэ төрлийг **хэрэглэхдээ**:

- Шинэ төрлийн объект буюу **instance** үүсгэх
- Тэд нар дээрээ **үйлдэл гүйцэтгэх**

Instance нь классын **тухайн тохиолдол** бөгөөд өөрийн гэсэн утгуудыг атрибутуудаа хадгална

Coordinate класс

```
class Coordinate(object):
    def __init__(self, xval, yval):
        """ Координат нь x, y утгаас бүтнэ """
        self.x = xval
        self.y = yval
    # бусад функцууд...
    def to_origin(self):
        """ self.x, self.y-н утгуудыг 0 болгоно """
        self.x = 0
        self.y = 0
```

Классаар класс байгуулах

- Жишээ: Координатаар тойрог бүтээх
- Бид **2 data attribute** хэрэглэнэ
  - Coordinate төрлийн объектоор тойргийн төв
  - int төрлийн объектоор тойргийн радиус

Circle класс тодорхойлох, instance үүсгэх

Coordinate object

int

```
class Circle(object):
    def __init__(self, center, radius):
        self.center = center
        self.radius = radius
```

Data attribute-ууд параметртайгаа адил нэртэй байх албагүй

```
center = Coordinate(2, 2)
my_circle = Circle(center, 2)
```

Circle класс тодорхойлох, instance үүсгэх

```
class Circle(object):
    def __init__(self, center, radius):
        self.center = center
        self.radius = radius
    def is_inside(self, point):
        return point.distance(self.center) < self.radius
```

self бол Circle object

point бол Coordinate object

Coordinate object

Coordinate obj.-н method дуудах

Coordinate object

```
center = Coordinate(2, 2)
my_circle = Circle(center, 2)
p = Coordinate(1, 1)
print(my_circle.is_inside(p))
```

Circle төрлийн объектоос л дуудах боломжтой method

Circle object

Coordinate object

Онцгой method-ууд

```
>>> a = [3, 4]
>>> print(a) # print(str(a))
[3, 4]
>>> c = Coordinate(3, 4)
>>> print(c) # print(str(c))
<__main__.Coordinate object at 0x1048f00b0>
```

- Илүү ойлгомжтой мэдээлэл хэвлэх боломжтой юу?
- Python класст “__” тэмдэглэгээтэй тусгай **dunder** функцүүдийг (**d**ouble **under**score **method**) тодорхойлох боломжтой байдаг

Dunder method жишээ: __str__

```
class Coordinate(object):
    def __init__(self, xval, yval):
        self.x = xval
        self.y = yval
    # бусад функцууд...

def __str__(self):
    return "(X=" + str(self.x) + ";Y=" + str(self.y) + ")"

>>> c = Coordinate(3, 4)
>>> print(c) # c.__str__()
(X=3;Y=4)
```

str() функц рүү Coordinate төрлийн объектыг дамжуулж ажиллуулахад ажиллах method (__str__)

тэмдэгт мөрт хөрвүүлж байгаа тул **str төрлийн утга** буцаах ёстой

print(c) үед c.__str__() функц ажиллаж тэмдэгт мөр буцаана

Dunder method-ууд

- Эхлэл болон төгсгөлд **хоёр доогуур зураас** бичнэ.
- Зарим dunder method-ууд:

```
__add__(self, other)      → self + other
__sub__(self, other)      → self - other
__mul__(self, other)      → self * other
__truediv__(self, other) → self / other
__eq__(self, other)       → self == other
__lt__(self, other)       → self < other
__len__(self)             → len(self)
__str__(self)             → print(self)
__float__(self)           → float(self) төрөл хувиргалт (cast)
__pow__                   → self**other
```

гэх мэт...

Жишээ: Энгийн бутархай

```
class Fraction:
    def __init__(self, n, d):
        self.num = n
        self.denom = d
    def __str__(self):
        return str(self.num) + "/" + str(self.denom)


# Example:
a = Fraction(1,4)
b = Fraction(3,5)
print(a) # prints 1/4
print(b) # prints 3/5
print(a * b) # TypeError: unsupported operand type(s) for *:...
```

Жишээ: Энгийн бутархай

```
class SimpleFraction(object):
    def __init__(self, n, d):
        self.num = n
        self.denom = d
    .........
    def times(self, other):
        top = self.num * other.num
        bottom = self.denom * other.denom
        return top / bottom
```

float төрлийн утга буцаана

```
a = SimpleFraction(1,4)
b = SimpleFraction(3,5)
print(a.times(b)) # prints 0.15
```

```
class Fraction:
    def __init__(self, n, d):
        self.num = n
        self.denom = d
    .........
    def __mul__(self, other):
        top = self.num * other.num
        bottom = self.denom * other.denom
        return Fraction(top, bottom)
```

шинэ Fraction төрлийн объект буцаана

```
a = Fraction(1,4)
b = Fraction(3,5)
print(a * b) # prints 3/20
```

Өдий хүртэл бидний хэрэглэж ирсэн операторуудын цаана method-ууд байсан байна.

Жишээ нь:
`print, len, +, -, *, /, <, >, <=, >=, ==, !=`
гэх мэт

Яагаад OOP?

- Ижил зорилго, нэг утга санаатай **өгөгдөл**, түүн дээр хийгдэх **үйлдлүүд**ийг **багцална**.
  - Код эмх **цэгцтэй**, **модуль**члагдсан болно
  - Цааш хөгжүүлэх, **арчлах**ад хялбар
  - Энгийн объектууд дээр **суурилж**, цааш илүү цогц объект үүсгэхэд хялбар
- Задлах болон хийсвэрлэх ойлголтыг Python классаар хэрэгжүүлнэ

Удамшил (Inheritance)

Яагаад OOP, Класс?

- Бодит амьдрал дээрх объектуудыг дуурайх
- Ижил төрлийн өөр өөр объектыг бүлэглэх

Instance:
Jelly
1 years old
brown

Instance:
5 years old
brown

Instance:
Tiger
2 years old
brown

Instance:
Bean
0 years old
black

Instance:
2 years old
white

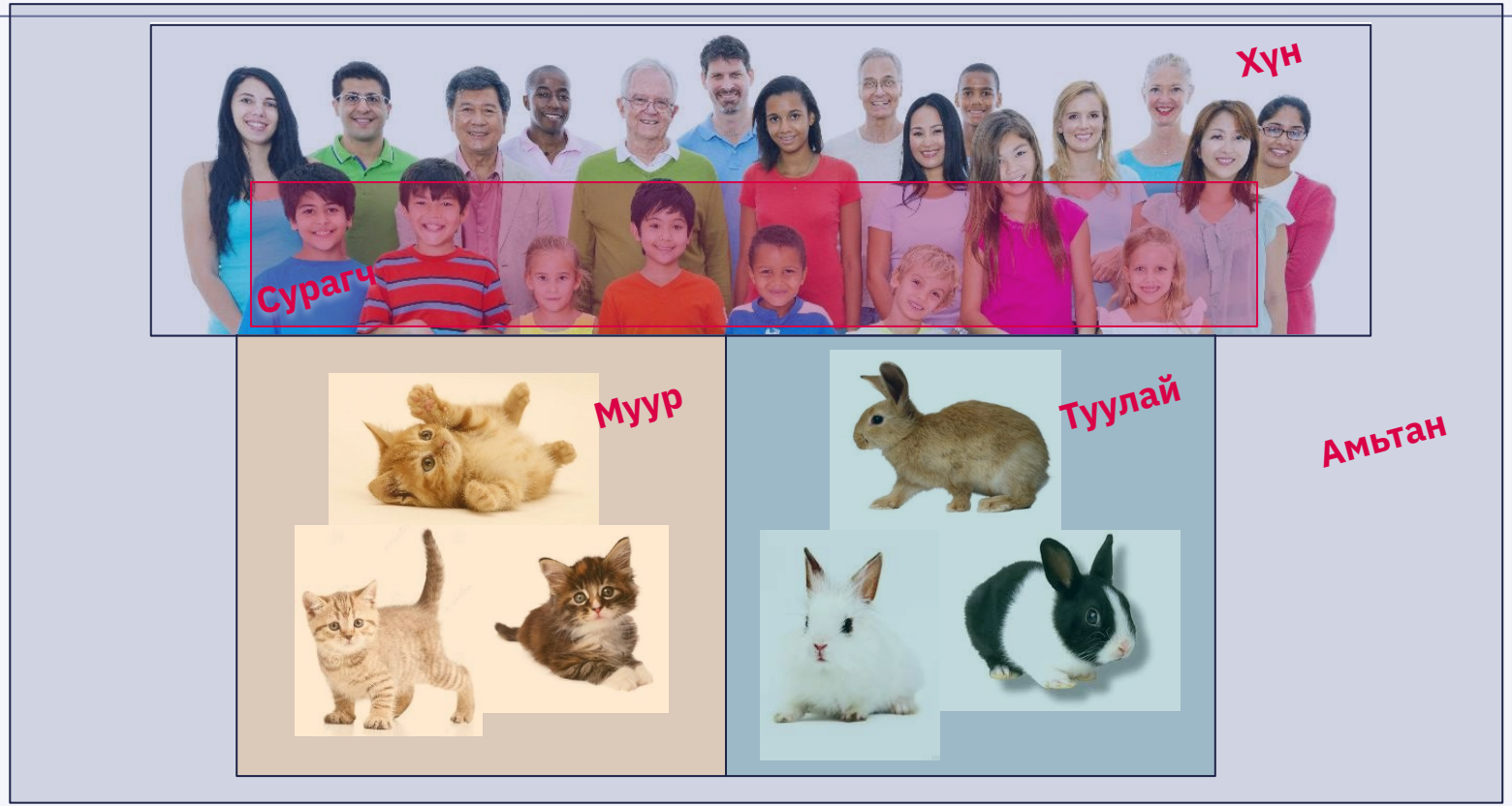
Instance:
1 years old
black/white

Яагаад OOP, Класс?

- Бодит амьдрал дээрх объектуудыг дуурайх
- Ижил төрлийн өөр өөр объектыг бүлэглэх

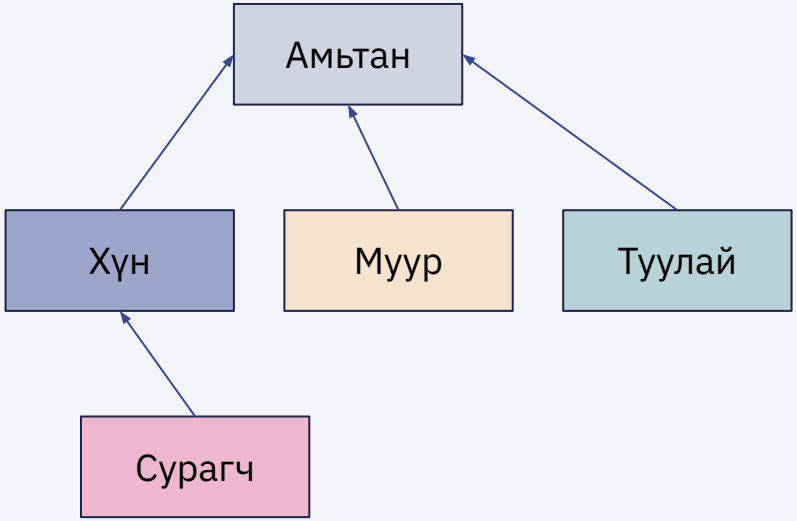
Нэг төрлийн бүх instance түгээмэл шинж чанар, зан авиртай

Шаталсан бүлэг



Шаталсан бүлэг

- **Эцэг класс** (superclass)
- **Хүүхэд класс** (subclass)
  - Эцгийн бүх шинжийг **удамшиж** авна
  - **Мэдээлэл нэмнэ**
  - **Үйлдэл нэмнэ**
  - Шинж чанарыг **дахин тодорхойлно**



Удамшил: Эцэг класс

```
class Animal(object):
    def __init__(self, age):
        self.age = age
        self.name = None

    def get_age(self):
        return self.age

    def get_name(self):
        return self.name

    def set_age(self, newage):
        self.age = newage

    def set_name(self, newname=""):
        self.name = newname

    def __str__(self):
        return "animal:"+str(self.name)+":"+str(self.age)
```

- Python-ы бүх класс нь object классаас удамшина
- object class нь бүх анхдагч үйлдлүүдийг агуулсан

Удамшил: Дэд класс - Cat

```
class Cat(Animal):
    def speak(self):
        print("meow")
    def __str__(self):
        return "cat:"+str(self.name)+":"+str(self.age)
```

Animal-н бүх атрибутыг удамшиж авна
__init__()
age, name
get_age(), get_name()
set_age(), set_name()
__str__()

Шинэ үйлдэл
speak method

__str__ -г дахин тодорхойлно (**override**)

- Cat нь Animal классаас удамшсан тул Animal-н бүх атрибут Cat-д орж ирнэ
- speak() гэдэг шинэ method нэмсэн
- __init__ байхгүйг анзаарч болно, Animal-н __init__ ажиллана

Аль method-г хэрэглэх вэ?

- Дэд классууд эцэг класстай **ижил нэртэй функц**тэй байж болно
- Эхлээд **өөрийн классын тодорхойлолт** дотор функцийг хайна
- Функц олдоогүй бол нэг **удамшлын шатлалаар яг дээр байх** класс дотор хайна (эцэг, өвөг эцэг г.м)
- Удамшлын модоор дээшилж байхдаа тааралдсан хамгийн эхний функцийг дуудна

Удамшил: Дэд класс - Person

```
class Person(Animal):
    def __init__(self, name, age):
        Animal.__init__(self, age)
        self.set_name(name)
        self.friends = []
    # бусад код...
```

Animal эцэг класс

__init__ method-г Person класст зориулж дахин тодорхойлсон

эцэг классын __init__ method-г дуудах

зөвхөн хүнд байх friends атрибут нэмэх

- `__init__` функцийг дахин тодорхойлсон:
  - Шинэ data attribute, объект үүсгэх үед өөр үйлдэл гүйцэтгэх шаардлагатай
  - Эцэг классын __init__-г классаар нь дамжуулан дуудна

Дэд класс эцгийн атрибутыг **хэрэглэнэ дахин тодорхойлно (override) нэмж тодорхойлно**

Атрибут нь өгөгдөл, method аль аль нь.